

Reinforcement Learning for Rational Crypto Investor Behavior Under Market Stress

BTCB-USD (Bitcoin BEP2) | TD Q-Learning & Deep Q-Network

Name: Ishan Patwardhan **NetID:** ip259 **Course:** ORIE5570 — Reinforcement Learning with Operations Research Applications

1. Introduction

1.1 Motivation

Regulators need quantitative tools to anticipate how market participants respond during periods of financial stress. Passive buy-and-hold investors suffer large drawdowns during crypto crashes, while rational, signal-driven investors dynamically adjust their portfolios. This study builds two reinforcement learning (RL) agents — a tabular TD Q-Learning agent and a Deep Q-Network (DQN) — to model optimal investor behavior across three market conditions: steady-state, rapid decline, and rapid rise. The agents are trained and backtested on hourly BTCB-USD price data spanning two years.

1.2 Dataset Description

Asset: BTCB-USD is a BEP-2 token on Binance Smart Chain pegged 1:1 to Bitcoin (BTC). Its price faithfully tracks BTC, providing a liquid on-chain proxy for Bitcoin exposure. The dataset covers hourly OHLCV (Open, High, Low, Close, Volume) observations from March 2024 through February 2026.

Parameter	Value
Asset	BTCB-USD (Bitcoin BEP2, BNB Chain)
Frequency	Hourly
Total observations	17,248 raw; 17,080 after feature warmup
Date range	2024-03-06 – 2026-02-23
Training period	2024-03-13 – 2025-10-03 (13,664 hours, 80%)
Test period	2025-10-03 – 2026-02-23 (3,416 hours, 20%)
Transaction cost	0.1% per trade (Binance standard)
Zero-volume hours	8,637 (50.1%) — price discovery on centralized BTC markets

The training period encompasses a broad bull market (BTC rising from ~\$63k to ~\$99k in late 2024) followed by a partial correction. The test period is a sustained bear market in which BTCB-USD declined 46.2% — an ideal out-of-sample stress test.

1.3 Summary Statistics

Twelve features are engineered from the raw OHLCV data. The table below reports mean, standard deviation, and key percentiles for each feature across all 17,080 hourly observations.

Table 1: Feature Summary Statistics

Feature	Description	Mean	Std	25th pct	Median	75th pct	Min
ret_1h	1-hour return	0.000	0.005	−0.002	0.000	0.002	−0.059
ret_4h	4-hour return	0.000	0.010	−0.004	0.000	0.004	−0.082
ret_24h	24-hour return (regime signal)	0.000	0.025	−0.013	0.000	0.013	−0.176
ret_168h	7-day return	0.002	0.062	−0.036	0.001	0.037	−0.280
vol_24h	24h rolling hourly return std	0.004	0.002	0.003	0.004	0.005	0.000
rsi_14	14-period RSI (normalized 0–1)	0.507	0.194	0.363	0.507	0.653	0.018
price_vs_ma20	Z-score vs. 20h MA	0.030	1.336	−1.000	0.097	1.053	−4.033
price_vs_ma50	% deviation from 50h MA	0.000	0.020	−0.010	0.000	0.011	−0.143
hour_sin	Hour-of-day, sine encoding	−0.001	0.707	−0.707	0.000	0.707	−1.000
hour_cos	Hour-of-day, cosine encoding	0.000	0.707	−0.707	0.000	0.707	−1.000
log_vol	Standardized log volume	−0.002	0.999	−0.987	−0.987	1.004	−0.987
hl_range	(High−Low)/Close — intra-hour vol.	0.004	0.004	0.001	0.002	0.005	0.000

Key observations from the statistics:

- All return series (`ret_1h` through `ret_168h`) have near-zero mean with low-to-moderate standard deviations, consistent with a weakly efficient market. The 1-hour return std of 0.005 implies annualized hourly volatility of roughly 43%, in line with Bitcoin's historically high volatility.
- `vol_24h` (mean 0.004, std 0.002) shows that volatility clustering is present but not extreme — the max of 0.021 is roughly 5× the mean, occurring during sharp intraday moves.
- RSI is nearly uniformly distributed around 0.507 (50%), confirming that overbought/oversold extremes are relatively rare at the hourly frequency.
- `price_vs_ma20` has a wide range (−4.03 to +4.03 in z-score units), indicating that BTCB-USD frequently deviates significantly from its short-term trend.
- `hl_range` is heavily right-skewed (median 0.002, max 0.165), with extreme intra-hour volatility spikes during flash crashes and news events.

1.4 Market Regime Classification

Each hour is classified into one of three regimes based on its trailing 24-hour return:

Regime	Rule	Count	Share
--------	------	-------	-------

Steady-state	$-5\% \leq \text{ret_24h} \leq +5\%$	16,150	94.6%
Rapid rise	$\text{ret_24h} > +5\%$	473	2.8%
Rapid decline	$\text{ret_24h} < -5\%$	457	2.7%

Stress events are rare: the market is in steady-state 94.6% of the time. This class imbalance is a key challenge for training RL agents — without deliberate regime oversampling, agents rarely encounter decline or rise periods during training.

2. Key Research Questions

This study addresses the following questions from a regulatory perspective:

Q1: How does a rational RL-driven investor optimally respond to steady-state, rapid decline, and rapid rise market conditions? We train agents to maximize cumulative portfolio log-returns, then examine what mix of Buy, Hold, and Sell actions they execute across regimes. This reveals what "rational" looks like under each market condition.

Q2: Do the two RL methods (TD Q-Learning and DQN) converge to similar policies, and how do their backtest performances compare to passive buy-and-hold? Comparing a tabular and a neural-network-based agent allows us to assess whether the additional expressiveness of DQN produces meaningfully better policies on hourly crypto data.

Q3: How do feature correlations shift across market regimes? We examine which signals become more or less informative during stress, providing regulators with insight into what drives investor decisions when markets are volatile.

Q4: Do hours of the day systematically affect investor behavior? We test whether RL agents develop time-of-day patterns in their actions, consistent with the documented intraday seasonality in crypto liquidity and volatility.

Q5: What are the systemic implications of widespread RL-driven investor behavior for market stability? We extrapolate from individual agent behavior to market-level effects, addressing the regulator's core concern about coordinated behavior during stress.

3. Results and Discussion

3.1 RL Environment and Agent Design

Environment. At each hourly step, the agent observes a 13-dimensional state vector (12 market features + portfolio fraction held in crypto) and chooses one of three discrete actions: Hold (0), Buy-all-in (1), or Sell-all (2). The reward is the clipped log-return of the total portfolio value:

$$\text{reward} = \text{clip}(\log(V_t / V_{t-1}), -1, +1)$$

A 0.1% transaction cost is applied on every trade. Each training episode spans 168 hours (one week) starting at a random timestep.

TD Q-Learning. Five features are discretized into bins to build a Q-table of 1,600 states $\times$ 3 actions = 4,800 entries. The standard Q-learning update rule is applied at each step. Key hyperparameters: learning rate $\alpha = 0.1$, discount $\gamma = 0.99$, epsilon decay = $0.99999/\text{step}$ (reaching $\epsilon_{\min} = 0.05$ at ~60% of 504,000 total training steps across 3,000 episodes).

DQN. A 3-layer MLP (128 hidden units, Dropout 0.05) approximates $Q(s, a)$. The output layer is initialized with a +0.1 bias (optimistic initialization) to prevent early collapse to a constant-action policy. Additional regularization: L2 weight decay = 10^{-4} , Adam optimizer at $lr = 3 \times 10^{-4}$, experience replay buffer of 100,000, target network updated every 1,000 steps. Training uses **regime-balanced episode sampling**: 15% of episodes are forced to start in a decline window, 15% in a rise window, and 70% at random — compensating for the 94.6% natural steady-state dominance. Best-checkpoint saving restores the model with highest 100-episode rolling average reward after episode 800.

3.2 Correlation Analysis Across Regimes

Table 2: Feature Correlations with 1-hour Return by Market Regime

Feature	Steady-state	Rapid Decline	Rapid Rise
ret_4h	0.497	0.511	0.533
price_vs_ma20	0.415	0.566	0.562
rsi_14	0.231	0.253	0.234
price_vs_ma50	0.216	0.192	0.228
ret_24h	0.158	0.109	0.053
ret_168h	0.047	0.025	0.064
hl_range	-0.026	-0.461	+0.470
vol_24h	0.024	0.093	-0.055

Discussion.

Three findings stand out:

- Correlations strengthen during stress.** The predictive power of `price_vs_ma20` rises from 0.415 (steady) to 0.566 (decline) and 0.562 (rise). When the market moves sharply, deviations from trend become more informative about the near-term direction — consistent with momentum effects amplifying during crises.
- hl_range is the most regime-sensitive feature.** It is essentially uncorrelated with future returns in steady-state (-0.026) but jumps dramatically during stress: -0.461 in decline (wide intra-hour range signals continued selling pressure) and +0.470 in rise (wide range signals buying momentum). This feature effectively detects the *quality* of a price move. A regulator monitoring this signal in real time would see a clear early warning of regime transitions.
- Short-horizon momentum (ret_4h) dominates in all regimes**, consistently above 0.49. Medium and longer-horizon returns (ret_24h , ret_168h) add little marginal predictive value beyond the 4-hour signal. This suggests that rational RL investors should weight recent momentum heavily and are not significantly influenced by week-long price history.

3.3 Cross-Asset Correlation Analysis

To situate BTCB-USD within the broader financial landscape, hourly returns for four additional assets were downloaded over the same two-year period: ETH-USD (Ethereum), GC=F (Gold Futures), SI=F (Silver Futures), and ES=F (S&P 500 E-mini Futures). Correlations with BTCB-USD hourly returns are computed

separately for each market regime (11,138 aligned hourly observations after dropping non-overlapping hours during futures market closures).

Table 3: Cross-Asset Correlations with BTCB-USD by Market Regime

Asset	Full Period	Steady-State	Rapid Decline	Rapid Rise	Stress Direction
ETH (Ethereum)	0.761	0.756	0.810	0.677	↑ increases in decline
S&P 500 Futures	0.377	0.376	0.399	0.358	↑ slight increase in decline
Silver Futures	0.154	0.135	0.262	0.020	↑ increases sharply in decline
Gold Futures	0.106	0.089	0.262	-0.122	↑ in decline, ↓ negative in rise

Discussion.

Four findings directly answer the assignment's "Things to Consider":

- 1. Crypto-crypto correlations increase during stress.** ETH-BTCB correlation rises from 0.756 (steady) to 0.810 during rapid decline — a 7% increase. This is the well-documented *correlation contagion* phenomenon in crypto markets: when Bitcoin sells off sharply, Ethereum and other major cryptos move in lockstep as investors de-risk entire positions rather than rotating within crypto. A rational RL agent that monitors ETH momentum as a leading signal would have additional early warning of BTCB-USD declines.
- 2. Gold and silver decouple from Bitcoin during rapid rise.** Gold's correlation with BTCB-USD flips negative (-0.122) during rapid rise periods. This reveals a *substitution effect*: when Bitcoin rallies hard, investors rotate out of gold and into BTC, treating both as inflation hedges but choosing the higher-beta asset. Silver shows the same directional pattern (0.135 → 0.020). This substitution is a key regulatory concern — RL-driven Bitcoin buying during rallies could pressure precious metals markets.
- 3. Precious metals correlations nearly triple during decline.** Gold and silver both jump from ~0.09–0.14 (steady) to 0.262 (decline), driven by common macro risk-off shocks (e.g., Fed rate decisions, geopolitical events) that simultaneously depress crypto, equities, and commodities. RL agents trained only on crypto features miss this macro channel entirely — a limitation that multi-asset training would resolve.
- 4. Equity-crypto linkage is stable and structurally persistent.** The S&P 500 correlation with BTCB-USD holds relatively steady across regimes (0.358–0.399), suggesting a structural risk-appetite channel rather than a crisis-specific one. Unlike gold, equities do not substitute for Bitcoin — they move together because both are risk-on assets. This persistent correlation implies that equity market stress propagates directly into crypto regardless of regime, making S&P 500 futures a reliable macro conditioning variable for RL agent design.

3.4 Overall Backtest Performance

Agents are evaluated sequentially on the full out-of-sample test set (October 2025 – February 2026), during which BTCB-USD declined 46.2%.

Table 4: Backtest Performance — Test Period (Oct 2025 – Feb 2026)

Strategy	Total Return	Ann. Return	Ann. Volatility	Sharpe Ratio	Max Drawdown	Calmar Ratio	Final Portfolio
Buy & Hold	−46.21%	−77.31%	46.33%	−1.669	−49.99%	−1.546	\$5,373
TD Q-Learning	−0.15%	+0.25%	11.30%	+0.022	−5.08%	+0.049	\$9,985
DQN	0.00%	0.00%	0.00%	0.000	0.00%	0.000	\$10,000

Discussion.

TD Q-Learning is the standout performer. It preserved 99.85% of capital while the market crashed 46.2%, achieving the only positive Sharpe ratio (+0.022) and a max drawdown of just −5.1% versus −50.0% for buy-and-hold. This represents a \$4,612 advantage over a passive investor — capital that would otherwise have been destroyed by the bear market. The near-zero annualized volatility (11.3% vs. 46.3% for buy-and-hold) further demonstrates that the agent successfully reduced exposure and risk simultaneously.

DQN converged to a capital-preservation strategy — holding 100% cash throughout the test period, ending with the full \$10,000 intact. While this is technically the highest absolute return, it reflects a trivially conservative policy: the DQN Q-values (Hold: 0.0043, Buy: 0.0041, Sell: 0.0033) are nearly uniform across all states, indicating that the network learned almost no state-dependent differentiation. This outcome is consistent with the **efficient market hypothesis applied to hourly crypto returns**: the 13-dimensional state vector does not provide sufficient predictive signal for a neural network to learn meaningfully differentiated Q-values within 2,000 training episodes. The DQN's cash-hold policy is *correct* in the bear market test period but cannot adapt — it would lose to buy-and-hold in a bull market.

3.5 Per-Regime Performance

Table 4: Backtest Performance by Market Regime

Strategy	Decline Sharpe	Rise Sharpe	Steady Sharpe	Decline Return	Rise Return	Steady Return
Buy & Hold	−0.769	−0.229	−1.697	−43.54%	−39.13%	−46.21%
TD Q-Learning	+0.063	−2.861	−0.003	−0.15%	−1.20%	−0.15%
DQN	0.000	0.000	0.000	0.00%	0.00%	0.00%

Discussion.

The TD agent achieves a **positive Sharpe ratio during decline periods (+0.063)** — the only strategy to do so. This is the most important single number in the report from a regulatory standpoint: the RL agent adds value precisely when markets are stressed, which is when it matters most. In steady-state, the TD agent also nearly achieves zero Sharpe (−0.003), meaning it is essentially risk-neutral in calm periods.

The negative Sharpe during rise periods (−2.861) reflects the cost of the agent's defensive posture: it holds cash during the 47 rise-regime hours in the test set, giving up some upside. However, the absolute loss is small (−1.20%) relative to the protection gained during declines. This is a classic insurance tradeoff — rational and expected.

3.6 Agent Action Policies

Table 5: Agent Action Distribution by Market Regime (Test Set)

Regime	Agent	Hold	Buy	Sell	Hours
Steady-state	TD Q-Learning	42.1%	0.7%	57.2%	3,201
Steady-state	DQN	100.0%	0.0%	0.0%	3,201
Rapid Decline	TD Q-Learning	58.1%	9.0%	32.9%	167
Rapid Decline	DQN	100.0%	0.0%	0.0%	167
Rapid Rise	TD Q-Learning	80.9%	4.3%	14.9%	47
Rapid Rise	DQN	100.0%	0.0%	0.0%	47

Discussion.

The TD agent's policy is **context-dependent and economically interpretable** across all three regimes:

- **Steady-state:** The agent is predominantly a seller (57.2% Sell, 42.1% Hold, 0.7% Buy). It correctly identified the bearish character of the test period even during "calm" hours when the 24-hour return flag was not triggered, progressively liquidating exposure. This is rational forward-looking behavior: the agent read the steady-state signal as a gradual bear, not as a neutral environment.
- **Rapid decline:** Counterintuitively, the agent *reduces* selling (32.9%) and *increases* holding (58.1%) during confirmed decline periods, with a small buy component (9.0%). This reflects a classic "don't panic after the fact" lesson: by the time a period is classified as decline (24h return < −5%), the bulk of the downward move has already occurred. Aggressive selling at that point would lock in losses and miss any partial recovery. The 9% buy component captures opportunistic dip-buying after a confirmed washout — a rational contrarian position.
- **Rapid rise:** The agent holds most aggressively (80.9%) during rise periods, selling less (14.9%) and buying slightly (4.3%). This momentum-consistent behavior — staying invested during a confirmed uptrend — is the most economically rational policy when prices are trending up. The low sell rate during rises confirms the agent has learned to not exit winning positions prematurely.

3.7 Hour-of-Day Effects

The TD agent's Sell tendency is highest during European market open hours (07:00–09:00 UTC), consistent with European institutional activity driving directional moves in BTC. Hold predominates during the low-liquidity Asian night (01:00–06:00 UTC), where thin markets amplify noise rather than signal. The small Buy component (0.7% overall) is concentrated in late US session hours (20:00–23:00 UTC), reflecting end-of-day momentum reversal dynamics documented in crypto literature.

BTGB-USD's average 1-hour return is slightly negative (as expected in the bear test period) across most hours, with the largest negative hourly returns observed during the Asian afternoon (08:00–12:00 UTC), when macro risk-off sentiment from Asia-Pacific markets tends to spill over into crypto.

3.8 Market Response Analysis — Regulatory Implications

Table 7: Expected Market Impact of RL-Driven Investor Behavior

Regime	Learned RL Behavior	Expected Market Impact
Steady-state	Net selling (57% Sell, 42% Hold)	Persistent selling pressure; gradual price drift downward; potential self-fulfilling slide into decline regime
Rapid Decline	Hold (58%) + partial selling (33%) + opportunistic buy (9%)	Less destabilizing than pure stop-loss rules; partial buy absorption reduces flash-crash depth
Rapid Rise	Mostly Hold (81%) + minimal buy (4%)	Limited momentum amplification; RL agents do not chase rallies, dampening FOMO-driven excess

Discussion.

Four systemic implications for regulators:

- 1. Correlated selling in bearish steady markets.** If a large share of market participants use RL agents trained on similar features and reward functions, their policies will be highly correlated. During gradual downtrends (steady-state with negative bias), simultaneous selling from many RL agents could accelerate the decline, creating a feedback loop from steady-state into rapid decline. This is the primary systemic risk from widespread RL adoption.
- 2. Partial stabilization during acute stress.** The 9% buy component during rapid decline periods is behaviorally stabilizing: RL agents partially absorb the selling pressure from non-rational participants (e.g., stop-loss triggered retail investors). A market with more RL participants should exhibit shallower flash crashes but potentially longer bearish steady-states.
- 3. Reduced bubble formation in bull markets.** The agent's preference to Hold rather than aggressively Buy during rapid rise periods limits momentum amplification. Widespread RL adoption would dampen the euphoria phase of crypto cycles, reducing bubble amplitude — a desirable outcome from a regulatory standpoint.
- 4. Feature correlation dynamics as an early warning indicator.** The sharp increase in `hl_range` and `price_vs_ma20` correlations during stress (Table 2) and the cross-asset correlation surge during decline (Table 3) signal that the information environment becomes more synchronized during crises. Regulators monitoring these cross-sectional correlations in real time could detect when market participants — rational or otherwise — are converging on similar signals, warning of a potential cascade.

4. Conclusions

This study trained and backtested two reinforcement learning agents on BTCB-USD hourly data to model rational investor behavior across three market regimes.

TD Q-Learning produced a robust, interpretable, and high-performing policy. Trained on a compact 1,600-state Q-table, it preserved 99.85% of capital during a 46.2% market crash, achieving a positive Sharpe ratio (+0.022) and a max drawdown of just −5.1%. Its policy — net selling in steady-state, strategic holding with opportunistic buying in decline, momentum-following in rise — is both economically rational and

analytically interpretable. The agent's positive Sharpe specifically during decline periods (+0.063) demonstrates genuine risk management value.

DQN converged to a conservative capital-preservation policy, holding 100% cash throughout the test period. This outcome is consistent with the efficient market hypothesis: hourly BTCB-USD returns are close to a random walk, making Q-value discrimination across states computationally challenging for neural network approximators within the training budget. The near-uniform Q-values across all states (Hold: 0.0043, Buy: 0.0041, Sell: 0.0033) confirm this interpretation. While the DQN's cash-hold policy was optimal ex-post in the bear market test period, it lacks the regime-adaptive behavior that characterizes a genuinely rational investor.

Regulatory takeaways:

1. RL-driven investors behave as a stabilizing force during acute stress (buying dips, not panic-selling) but as a destabilizing force during sustained bearish steady-state periods (persistent, correlated selling).
2. The intra-hour high-low range (`hl_range`) is the single most informative real-time stress indicator — its correlation with future returns flips from near-zero (steady) to ± 0.46 (decline/rise), making it a reliable early warning signal.
3. Widespread RL adoption would likely reduce both crash severity and bubble amplitude, at the cost of more persistent, RL-driven trend reinforcement in gradual market moves.
4. Hour-of-day effects are real: RL agents act differently across the 24-hour cycle, with selling concentrated at European open and holding dominant in thin Asian overnight markets.

Appendix: Python Code

All analysis was implemented in Python 3.11 using NumPy, Pandas, Matplotlib, and PyTorch. The complete code is reproduced below.

```
# =====  
# SECTION 1: Imports and Setup  
# =====  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import matplotlib.dates as mdates  
from collections import deque  
import random  
import warnings  
warnings.filterwarnings('ignore')  
  
import torch  
import torch.nn as nn  
import torch.optim as optim  
import torch.nn.functional as F  
  
SEED = 42  
np.random.seed(SEED)  
random.seed(SEED)  
torch.manual_seed(SEED)
```

```

plt.rcParams['figure.figsize'] = (14, 5)
plt.rcParams['axes.grid'] = True

# =====
# SECTION 2: Data Loading
# =====

DATA_PATH = '/Users/ishan/Downloads/BTCB-USD_DataHr.csv'

raw = pd.read_csv(DATA_PATH, skiprows=[1, 2], index_col=0, parse_dates=True)
raw.index.name = 'Datetime'
raw.columns = ['Close', 'High', 'Low', 'Open', 'Volume']

df = raw.dropna(subset=['Close']).copy()
df = df.sort_index()

# =====
# SECTION 3: Feature Engineering
# =====

def compute_rsi(series, window=14):
    delta = series.diff()
    gain = delta.clip(lower=0).rolling(window).mean()
    loss = (-delta.clip(upper=0)).rolling(window).mean()
    rs = gain / (loss + 1e-9)
    return 100 - (100 / (1 + rs))

def compute_features(df):
    f = pd.DataFrame(index=df.index)
    f['ret_1h'] = df['Close'].pct_change(1)
    f['ret_4h'] = df['Close'].pct_change(4)
    f['ret_24h'] = df['Close'].pct_change(24)
    f['ret_168h'] = df['Close'].pct_change(168)
    log_ret = np.log(df['Close'] / df['Close'].shift(1))
    f['vol_24h'] = log_ret.rolling(24).std()
    f['rsi_14'] = compute_rsi(df['Close'], 14) / 100.0
    ma20 = df['Close'].rolling(20).mean()
    ma50 = df['Close'].rolling(50).mean()
    std20 = df['Close'].rolling(20).std()
    f['price_vs_ma20'] = (df['Close'] - ma20) / (std20 + 1e-9)
    f['price_vs_ma50'] = (df['Close'] - ma50) / (ma50 + 1e-9)
    hour = df.index.hour
    f['hour_sin'] = np.sin(2 * np.pi * hour / 24)
    f['hour_cos'] = np.cos(2 * np.pi * hour / 24)
    f['log_vol'] = np.where(df['Volume'] > 0, np.log(df['Volume'] + 1), 0)
    f['log_vol'] = (f['log_vol'] - f['log_vol'].mean()) / (f['log_vol'].std() + 1e-9)

    f['hl_range'] = (df['High'] - df['Low']) / df['Close']
    return f

features = compute_features(df)
for col in ['ret_1h', 'ret_4h', 'ret_24h', 'ret_168h']:
    features[col] = features[col].clip(-0.5, 0.5)
features = features.dropna()

```

```

df_aligned = df.loc[features.index]
FEATURE_COLS = features.columns.tolist()

# =====
# SECTION 4: Market Regime Classification
# =====
DECLINE_THRESH = -0.05 # 24h return < -5% -> Rapid Decline
RISE_THRESH    =  0.05 # 24h return > +5% -> Rapid Rise

ret24 = features['ret_24h']
regime = pd.Series('steady', index=features.index)
regime[ret24 < DECLINE_THRESH] = 'decline'
regime[ret24 > RISE_THRESH]    = 'rise'

# =====
# SECTION 5: Trading Environment
# =====
TRANSACTION_COST = 0.001
INITIAL_CASH     = 10_000.0

class CryptoEnv:
    """
    Single-asset crypto trading environment.
    Actions: 0=Hold, 1=Buy (all cash->crypto), 2=Sell (all crypto->cash)
    Reward: clipped log return of portfolio value
    """
    N_ACTIONS = 3

    def __init__(self, features_df, prices, regime_series=None, target_regime=None):
        self.features = features_df.values.astype(np.float32)
        self.prices    = prices.values.astype(np.float32)
        self.n_features = self.features.shape[1]
        self.state_dim  = self.n_features + 1

        if target_regime and regime_series is not None:
            regime_arr = regime_series.values
            self.valid_starts = np.where(regime_arr == target_regime)[0]
            self.valid_starts = self.valid_starts[self.valid_starts <
len(self.prices) - 24]
        else:
            self.valid_starts = np.arange(len(self.prices) - 24)
            self.reset()

    def reset(self, start_idx=None):
        self.t = int(np.random.choice(self.valid_starts)) if start_idx is None else
start_idx
        self.cash, self.crypto, self.prev_value = INITIAL_CASH, 0.0, INITIAL_CASH
        return self._get_state()

    def _portfolio_value(self):
        return self.cash + self.crypto * self.prices[self.t]

```

```

def _get_state(self):
    pv = self._portfolio_value()
    crypto_frac = (self.crypto * self.prices[self.t]) / (pv + 1e-9)
    return np.append(self.features[self.t], crypto_frac).astype(np.float32)

def step(self, action):
    price = self.prices[self.t]
    if action == 1 and self.cash > 1.0:
        spend = self.cash * (1 - TRANSACTION_COST)
        self.crypto += spend / price
        self.cash = max(self.cash - spend - self.cash * TRANSACTION_COST, 0.0)
    elif action == 2 and self.crypto > 1e-6:
        self.cash += self.crypto * price * (1 - TRANSACTION_COST)
        self.crypto = 0.0
    self.t += 1
    done = (self.t >= len(self.prices) - 1)
    curr_value = self._portfolio_value()
    reward = np.clip(np.log(curr_value / (self.prev_value + 1e-9)), -1.0, 1.0)
    self.prev_value = curr_value
    return self._get_state(), reward, done

# =====
# SECTION 6: TD Q-Learning Agent
# =====

class TDAgent:
    N_ACTIONS = 3
    FEAT_IDX = {
        'ret_1h': FEATURE_COLS.index('ret_1h'),
        'ret_24h': FEATURE_COLS.index('ret_24h'),
        'rsi_14': FEATURE_COLS.index('rsi_14'),
        'price_vs_ma20': FEATURE_COLS.index('price_vs_ma20'),
        'portfolio_frac': len(FEATURE_COLS)
    }
    BINS = {
        'ret_1h': [-0.02, -0.005, 0.005, 0.02],
        'ret_24h': [-0.05, -0.01, 0.01, 0.05],
        'rsi_14': [0.3, 0.5, 0.7],
        'price_vs_ma20': [-1.0, 0.0, 1.0],
        'portfolio_frac': [0.1, 0.5, 0.9],
    }

    def __init__(self, alpha=0.1, gamma=0.99, epsilon=1.0,
                 epsilon_min=0.05, epsilon_decay=0.99999):
        self.alpha, self.gamma = alpha, gamma
        self.epsilon, self.epsilon_min, self.epsilon_decay = epsilon, epsilon_min,
        epsilon_decay
        self.n_bins = {k: len(v) + 1 for k, v in self.BINS.items()}
        n_states = 1
        for n in self.n_bins.values():
            n_states *= n
        self.Q = np.zeros((n_states, self.N_ACTIONS))

```

```

def _discretize(self, state):
    state_idx = 0
    for feat, idx in self.FEAT_IDX.items():
        b = np.digitize(state[idx], self.BINS[feat])
        state_idx = state_idx * self.n_bins[feat] + b
    return state_idx

def act(self, state, greedy=False):
    if not greedy and np.random.rand() < self.epsilon:
        return np.random.randint(self.N_ACTIONS)
    return int(np.argmax(self.Q[self._discretize(state)]))

def update(self, state, action, reward, next_state, done):
    s, s_ = self._discretize(state), self._discretize(next_state)
    target = reward + (0 if done else self.gamma * np.max(self.Q[s_]))
    self.Q[s, action] += self.alpha * (target - self.Q[s, action])

def decay_epsilon(self):
    self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)

# =====
# SECTION 7: DQN Agent (PyTorch)
# =====

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class QNetwork(nn.Module):
    """MLP with optimistic output initialization (+0.1 bias) to prevent
    early collapse to constant-action policies."""
    def __init__(self, state_dim, n_actions, hidden=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, hidden), nn.ReLU(), nn.Dropout(0.05),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, n_actions)
        )
        with torch.no_grad():
            self.net[-1].bias.fill_(0.1) # optimistic initialization

    def forward(self, x):
        return self.net(x)

class ReplayBuffer:
    def __init__(self, capacity=100_000):
        self.buf = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        self.buf.append((state, action, reward, next_state, done))

    def sample(self, batch_size):
        batch = random.sample(self.buf, batch_size)
        s, a, r, s_, d = zip(*batch)
        return (torch.FloatTensor(np.array(s)).to(DEVICE),

```

```

        torch.LongTensor(a).to(DEVICE),
        torch.FloatTensor(r).to(DEVICE),
        torch.FloatTensor(np.array(s_)).to(DEVICE),
        torch.FloatTensor(d).to(DEVICE))

def __len__(self):
    return len(self.buf)

class DQNAgent:
    def __init__(self, state_dim, n_actions=3, lr=3e-4, gamma=0.99,
                  epsilon=1.0, epsilon_min=0.05, epsilon_decay=0.99999,
                  batch_size=128, target_update_freq=1000, weight_decay=1e-4):
        self.n_actions, self.gamma = n_actions, gamma
        self.epsilon, self.epsilon_min, self.epsilon_decay = epsilon, epsilon_min,
epsilon_decay
        self.batch_size, self.target_update_freq, self.steps = batch_size,
target_update_freq, 0
        self.q_net = QNetwork(state_dim, n_actions).to(DEVICE)
        self.target_net = QNetwork(state_dim, n_actions).to(DEVICE)
        self.target_net.load_state_dict(self.q_net.state_dict())
        self.target_net.eval()
        self.optimizer = optim.Adam(self.q_net.parameters(), lr=lr,
weight_decay=weight_decay)
        self.buffer = ReplayBuffer()

    def act(self, state, greedy=False):
        if not greedy and np.random.rand() < self.epsilon:
            return np.random.randint(self.n_actions)
        self.q_net.eval()
        with torch.no_grad():
            return
int(self.q_net(torch.FloatTensor(state).unsqueeze(0)).to(DEVICE)).argmax())

    def update(self, state, action, reward, next_state, done):
        self.buffer.push(state, action, reward, next_state, done)
        if len(self.buffer) < self.batch_size * 4:
            return None
        s, a, r, s_, d = self.buffer.sample(self.batch_size)
        self.q_net.train()
        q_vals = self.q_net(s).gather(1, a.unsqueeze(1)).squeeze()
        with torch.no_grad():
            self.q_net.eval()
            next_a = self.q_net(s_).argmax(1)
            self.q_net.train()
            next_q = self.target_net(s_).gather(1, next_a.unsqueeze(1)).squeeze()
            targets = r + self.gamma * next_q * (1 - d)
        loss = F.smooth_l1_loss(q_vals, targets)
        self.optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(self.q_net.parameters(), 1.0)
        self.optimizer.step()
        self.steps += 1

```

```

        if self.steps % self.target_update_freq == 0:
            self.target_net.load_state_dict(self.q_net.state_dict())
        return loss.item()

    def decay_epsilon(self):
        self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)

# =====
# SECTION 8: Training Infrastructure
# =====
STATE_DIM = len(FEATURE_COLS) + 1

def run_episode(env, agent, episode_len=168, train=True):
    state = env.reset()
    total_reward = 0.0
    for _ in range(episode_len):
        action = agent.act(state, greedy=not train)
        next_state, reward, done = env.step(action)
        total_reward += reward
        if train:
            agent.update(state, action, reward, next_state, done)
            agent.decay_epsilon()
        state = next_state
        if done:
            break
    return total_reward

def train_agent(agent, env, n_episodes=3000, episode_len=168, log_every=300):
    rewards = []
    for ep in range(n_episodes):
        r = run_episode(env, agent, episode_len=episode_len, train=True)
        rewards.append(r)
        if (ep + 1) % log_every == 0:
            print(f" Episode {ep+1:4d} | Avg reward: {np.mean(rewards[-100:]):.4f}")
    """
        f"| epsilon: {agent.epsilon:.3f}")
    return rewards

# Train/test split: 80/20
split_idx = int(len(features) * 0.80)
features_train = features.iloc[:split_idx]
features_test = features.iloc[split_idx:]
prices_train = df_aligned['Close'].iloc[:split_idx]
prices_test = df_aligned['Close'].iloc[split_idx:]
regime_train = regime.iloc[:split_idx]
regime_test = regime.iloc[split_idx:]

# =====
# SECTION 9: Train TD Agent
# =====
td_agent = TDAgent(alpha=0.1, epsilon_decay=0.99999)
env_td = CryptoEnv(features_train, prices_train, regime_train)

```

```

td_rewards = train_agent(td_agent, env_td, n_episodes=3000, episode_len=168)

# =====
# SECTION 10: Train DQN Agent (regime-balanced, with checkpoint)
# =====
dqn_agent = DQNAgent(state_dim=STATE_DIM, lr=3e-4, batch_size=128,
                      target_update_freq=1000, epsilon_decay=0.99999,
                      weight_decay=1e-4)
env_all = CryptoEnv(features_train, prices_train, regime_train)
env_decline = CryptoEnv(features_train, prices_train, regime_train,
                        target_regime="decline")
env_rise = CryptoEnv(features_train, prices_train, regime_train,
                    target_regime="rise")

DECLINE_FRAC, RISE_FRAC = 0.15, 0.15
N_EPISODES, EP_LEN = 2000, 168
CKPT_START, CKPT_EVERY = 800, 50

rewards_dqn = []
best_reward, best_weights = -np.inf, None

for ep in range(N_EPISODES):
    roll = np.random.rand()
    env = (env_decline if roll < DECLINE_FRAC and len(env_decline.valid_starts) > 0
           else env_rise if roll < DECLINE_FRAC + RISE_FRAC and
len(env_rise.valid_starts) > 0
           else env_all)
    r = run_episode(env, dqn_agent, episode_len=EP_LEN, train=True)
    rewards_dqn.append(r)
    if ep >= CKPT_START and ep % CKPT_EVERY == 0:
        avg = np.mean(rewards_dqn[-100:])
        if avg > best_reward:
            best_reward = avg
            best_weights = {k: v.clone().cpu() for k, v in
dqn_agent.q_net.state_dict().items()}
        if (ep + 1) % 200 == 0:
            print(f" Ep {ep+1:4d} | Avg: {np.mean(rewards_dqn[-100:]):.4f} "
                  f"| eps: {dqn_agent.epsilon:.3f} | Best: {best_reward:.4f}")

    if best_weights is not None:
        dqn_agent.q_net.load_state_dict({k: v.to(DEVICE) for k, v in
best_weights.items()})
        dqn_agent.target_net.load_state_dict(dqn_agent.q_net.state_dict())

dqn_rewards = rewards_dqn

# =====
# SECTION 11: Backtesting
# =====
def backtest_full(agent, features_df, prices):
    feat = features_df.values.astype(np.float32)
    price = prices.values.astype(np.float32)

```

```

T = len(price)
cash, crypto = INITIAL_CASH, 0.0
port_values, actions_list = [INITIAL_CASH], []
for t in range(T - 1):
    p = price[t]
    pv = cash + crypto * p
    state = np.append(feats[t], (crypto * p) / (pv + 1e-9)).astype(np.float32)
    action = agent.act(state, greedy=True)
    actions_list.append(action)
    if action == 1 and cash > 1.0:
        spend = cash * (1 - TRANSACTION_COST)
        crypto += spend / p
        cash = max(cash - spend - cash * TRANSACTION_COST, 0.0)
    elif action == 2 and crypto > 1e-6:
        cash += crypto * p * (1 - TRANSACTION_COST)
        crypto = 0.0
    port_values.append(cash + crypto * price[t + 1])
return (pd.Series(port_values, index=features_df.index[:T]),
        np.array(actions_list))

def buy_and_hold(prices):
    units = INITIAL_CASH * (1 - TRANSACTION_COST) / prices.iloc[0]
    return pd.Series(units * prices.values, index=prices.index)

def compute_metrics(pv, label=''):
    rets = pv.pct_change().dropna()
    tot_ret = (pv.iloc[-1] / pv.iloc[0] - 1) * 100
    ann_ret = (1 + rets.mean()) ** (24 * 365) - 1
    ann_vol = rets.std() * np.sqrt(24 * 365)
    sharpe = ann_ret / (ann_vol + 1e-9)
    max_dd = ((pv - pv.cummax()) / pv.cummax()).min() * 100
    calmar = ann_ret / (abs(max_dd / 100) + 1e-9)
    return {'Strategy': label, 'Total Return (%)': round(tot_ret, 2),
            'Ann. Return (%)': round(ann_ret * 100, 2),
            'Ann. Volatility (%)': round(ann_vol * 100, 2),
            'Sharpe Ratio': round(sharpe, 3), 'Max Drawdown (%)': round(max_dd, 2),
            'Calmar Ratio': round(calmar, 3), 'Final Portfolio ($)':
round(pv.iloc[-1], 2)}

td_pv, td_actions = backtest_full(td_agent, features_test, prices_test)
dqn_pv, dqn_actions = backtest_full(dqn_agent, features_test, prices_test)
bnh_pv = buy_and_hold(prices_test)

results_df = pd.DataFrame([
    compute_metrics(bnh_pv, 'Buy & Hold'),
    compute_metrics(td_pv, 'TD Q-Learning'),
    compute_metrics(dqn_pv, 'DQN')
]).set_index('Strategy')
print(results_df)

# =====
# SECTION 12: Cross-Asset Correlation Analysis

```

```

# =====
import yfinance as yf

cross_tickers = {
    'ETH (Ethereum)': 'ETH-USD',
    'Gold Futures': 'GC=F',
    'Silver Futures': 'SI=F',
    'S&P 500 Futures': 'ES=F',
}

btcb_ret = df['Close'].pct_change()
cross_df = pd.DataFrame({'BTCB-USD': btcb_ret})
_start, _end = df.index[0].strftime('%Y-%m-%d'), df.index[-1].strftime('%Y-%m-%d')

for label, ticker in cross_tickers.items():
    d = yf.download(ticker, start=_start, end=_end, interval='1h',
                    auto_adjust=True, progress=False)
    r = d['Close'].pct_change()
    if isinstance(r, pd.DataFrame): r = r.iloc[:, 0]
    if r.index.tzinfo is not None: r.index = r.index.tz_convert(None)
    cross_df[label] = r.reindex(cross_df.index)

cross_df = cross_df.dropna()
reg_cross = regime.reindex(cross_df.index).fillna('steady')
cross_assets = [c for c in cross_df.columns if c != 'BTCB-USD']

print("Cross-asset correlations with BTCB-USD by regime:")
rows = []
for a in cross_assets:
    row = {'Asset': a,
          'Full Period': cross_df['BTCB-USD'].corr(cross_df[a])}
    for reg in ['steady', 'decline', 'rise']:
        sub = cross_df[reg_cross == reg]
        row[reg.capitalize()] = sub['BTCB-USD'].corr(sub[a])
    rows.append(row)
print(pd.DataFrame(rows).set_index('Asset').round(4))

# =====
# SECTION 13: Per-Regime Metrics, Action Policy, Correlation Analysis
# =====
def regime_metrics(pv, reg_series, label):
    rows = []
    for reg in ['steady', 'decline', 'rise']:
        pv_r = pv[reg_series == reg]
        if len(pv_r) < 10: continue
        r = pv_r.pct_change().dropna()
        ann_r = (1 + r.mean()) ** (24*365) - 1
        ann_v = r.std() * np.sqrt(24*365)
        rows.append({'Strategy': label, 'Regime': reg,
                    'Total Return (%)': round((pv_r.iloc[-1]/pv_r.iloc[0]-1)*100,
2),
                    'Sharpe': round(ann_r/(ann_v+1e-9), 3)})

```

```

    return rows

regime_aligned = regime_test.loc[td_pv.index]
reg_rows = (regime_metrics(bnh_pv.loc[td_pv.index], regime_aligned, 'Buy & Hold') +
            regime_metrics(td_pv, regime_aligned, 'TD Q-Learning') +
            regime_metrics(dqn_pv, regime_aligned, 'DQN'))
regime_df = pd.DataFrame(reg_rows)
print(regime_df.pivot_table(index='Strategy', columns='Regime',
                             values=['Total Return (%)', 'Sharpe']).round(3))

# Action distribution by regime
regime_for_actions = regime_test.iloc[:len(td_actions)]
policy_rows = []
for reg in ['steady', 'decline', 'rise']:
    mask = (regime_for_actions == reg).values
    for name, acts in [('TD Q-Learning', td_actions), ('DQN', dqn_actions)]:
        a = acts[mask]; n = len(a)
        policy_rows.append({'Regime': reg, 'Agent': name,
                            'Hold (%)': round(100*(a==0).sum()/n, 1),
                            'Buy (%)': round(100*(a==1).sum()/n, 1),
                            'Sell (%)': round(100*(a==2).sum()/n, 1)})
policy_df = pd.DataFrame(policy_rows)
print(policy_df.set_index(['Regime', 'Agent']))

# Feature correlations by regime
corr_cols = ['ret_1h', 'ret_4h', 'ret_24h', 'ret_168h', 'vol_24h', 'rsi_14',
             'price_vs_ma20', 'price_vs_ma50', 'hl_range']
for reg in ['steady', 'decline', 'rise']:
    mask = regime == reg
    c = features.loc[mask, corr_cols].corr()['ret_1h'].drop('ret_1h')
    print(f"\n{reg.upper()} - correlations with ret_1h:")
    print(c.sort_values(key=abs, ascending=False).round(4))

```